

1

First we had to think about how we would make our program recognize if a file exist and if it is accessible. We had first decided to do this with the command "access()", since this command makes us able to see, if a file exists, is readable, is write-able and if you have permission to execute it. We did end up changing our code and using the command `f open()` instead. "`f open()`" opens the file, if it is able to, and otherwise saves an "errno" to indicate the error, which we have then printed, using a function "print error". This way we can differentiate between a file not existing and a file not being available.

Then we went on to take care of errors that could happen with input from users. We did this by using if-statements. We should be able to handle if a user doesn't give the correct amount of arguments, so we made an if-statement that checks if a user puts in less than 2 arguments. If that is the case, then we print the message "Usage: file path", which gives a hint to what the user needs to do if they want to use the program. On the other hand the user could also write too many arguments. This gets handled by the next if-statement, which checks if the user put in more than 2 arguments, in which case we print "Wrong number of arguments" to again, give the user a nudge in the right direction.

Next we had to be able to differentiate between different files. We have chosen to implement the 3 basic types of files; Empty, ASCII text and Data. Why will be explained later under "Limitations and potential problems". We wanted to check first if a file was empty, since that would automatically rule out the two other file types. We can spot if a file is empty if it has exactly 0 bytes. Because we use `f read()` to read the first byte in the file, we use an if-statement that checks if there is a byte to read, and if not then declares the file type as empty.

The next thing we want to check is if the file is an ASCII text file. We do this first with a while-loop. We use the while-loop to read through the file, and with an if-statement, we check if all the bytes are in between 1 and 127. If they are, then it is an ASCII file. The while-loop stops when there is no more to read. This stops the empty files from entering the while-loop, since there isn't anything to read. If any of the bytes read doesn't match up with the ASCII text requirements, then it goes on to data and breaks out of the while loop.

Since data type should catch any type of file that isn't Empty and isn't an ASCII text, then we chose to use that in an else-statement, to catch whatever empty and ASCII doesn't pick up. This is the last place in our code, where the file type can be changed before being printed to the terminal.

2

We have set up a total of 12 test cases for the implementation of our file types; ASCII text, Data and Empty in order to test the types of our files. We then ran the tests with `bash test.sh`, where we were testing to validate whether or not our input type is the same as our expected type using the commando `-diff`. We made sure to test different lengths of the files content, to make sure it reads the entire way through, and doesn't just stop.

A problem that occurred while testing was that all the tests “failed”, even though, the files we were comparing had the same type. This could indicate that there might have been a problem with the script itself.

2.1

We were working with a non-readable file, “hemmelig fil”, which is unable to be accessed, read or tested, due to the fact that it is a non-readable file and therefore it isn’t possible to test it, which is why it isn’t included in the test files. We have “tested” it by creating a non-readable file and running it through our program, validating that its inaccessible, and that it returns the correct errorcode.

We did this using the commands:

```
$ make file  
$ ./file hemmelig_fil
```

which returned :

```
hemmelig_fil: cannot determine (Permission denied)
```

The file was made non-readable through the file’s properties, since we couldn’t get chmod to work as intended using:

```
$ chmod -r hemmelig_fil
```

We used the -diff command to compare our three different file types. This command is very particular about what is written and what it will handle as a “failed” test, since it interprets the files we were comparing as different, even though the type of the two files we were testing were the same.

We have not included test for the upper and lower limits of our files, meaning the length of bytes in our files, since we in our implementation differentiated between the length of bytes, in order to tell which type of file we had.

3

As mentioned earlier we have only implemented the file-types; Empty, ASCII text and Data, and not the types ISO-8859-1 text and UTF-8 text. This was due to time constraints. If we were to implement it, we think it could be done in somewhat the same way as filtering ASCII. For ISO-8859-1 text by checking if the file is composed of ISO-8859-1-like bytes, i.e. ASCII-like bytes and decimal values from 160-255 and for UTF-8-Unicode text by checking if it is composed of UTF-8-like characters. This would be more difficult, as UTF-8 encodes the characters in one to four bytes.

4

In this assignment we have implemented a program that can differentiate between three different types of files; Empty, ASCII text and Data. We have also implemented handling of user errors, such as the wrong amount of inputs, files

that don't exist and files we don't have permission to open. We have also tested the implementation and have come across an unexpected error, even though our actual output didn't differ from our expected output. We have used the tests to conclude that our implementation works as intended.